

W8D4

Task 1 — GameShell · Task 2 — Attacco brute-force a SSH

Sommario

Il Task 1 è il completamento di GameShell, un gioco a missioni per imparare i comandi della shell Linux. Tutte le missioni sono state completate. Il Task 2 riguarda l'attacco brute-force a un servizio SSH con una prova in laboratorio isolato tra una VM Kali (attaccante) e una VM Metasploitable 2 (bersaglio).

La Parte 1 è una guida operativa ai comandi della shell. La Parte 2 descrive l'ambiente, il funzionamento dell'attacco, la metodologia seguita, l'analisi dei risultati e il rilevamento tramite i log.

Parte 1 — Task 1: GameShell

Guida operativa ai comandi

Navigazione: pwd, ls, cd

pwd stampa la directory corrente; ls elenca il contenuto (-l dettagli, -a inclusi i nascosti); cd cambia directory (.. sale, ~ home, - precedente).

```
pwd ; ls -la ; cd ~/Documenti ; cd ..
```



Gestione dei file: cp, mv, rm

cp copia (-r per le cartelle), mv sposta o rinomina, rm elimina (-r ricorsivo, -i con conferma). In shell non esiste un cestino: rm è definitivo.

```
cp a.txt b.txt ; mv b.txt archivio/ ; rm -i vecchio.txt
```

Wildcard (glob): *, ?, []

La shell espande i jolly prima di eseguire: * qualsiasi sequenza, ? un carattere, [abc] uno tra quelli.

```
ls *.txt ; rm foto?.png
```

Alias

Scorciatoia per un comando.

```
alias ll='ls -la'
```



Visualizzazione: cat, head, tail

cat intero file; head -n prime righe; tail -n ultime; tail -f segue in tempo reale.

```
cat file.txt ; head -n 5 log.txt
```

Permessi: ls -l, chmod

ls -l mostra i permessi (es. drwxr-xr-x): 1° carattere tipo (d/-), poi tre terzine rwx per utente/gruppo/altri. chmod li modifica, in forma simbolica (u+x) od ottale (700). Su una cartella la x serve per attraversarla (cd); la sola r consente solo di leggerne l'elenco. I file nascosti (nome con .) si vedono con ls -a.

```
ls -la ; chmod u+rx Cartella ; chmod 700 Cartella
```

Pipe: |

Collega lo stdout di un comando allo stdin del successivo.

```
grep -v "PAGATO" * | wc -l
```

Redirezioni: >, >>, <, 2>, /dev/null

> stdout su file (sovrascrive), >> in coda, < stdin da file, 2> stderr, /dev/null scarta. Molti programmi (incluso gsh check) leggono la risposta da stdin: con < gliela si passa da file.

```
cmd > out.txt 2> err.txt ; gsh check < chiave.txt
```

Filtri di testo: grep, wc

```
Manda i messaggi di errore (stderr) al file.
(Il file viene sovrascritto.)

/dev/null
File "virtuale" che è universalmente vuoto: qualunque cosa mandata ad esso, scompare.

Spiegazioni

Ogni comando può stampare messaggi sullo schermo in 2 modi:
- stampa standard ("stdout")
- stampa di errori ("stderr")
Come predefinito entrambi i tipi di messaggio sono mostrati a schermo.
È tuttavia possibile reindirizzare entrambi to file indipendenti.

)= (=@=()

~/Castello/Palazzo_principale/Biblioteca/Ufficio_di_Merlino
[mission 35] $ ls
candela
cassetto/
grimorio_BARafVAGQXoeJHGhIZtCLT
grimorio_BaZEKSAVEyYnnpomFUDVishPiS
grimorio_bA2JufyVlSnJLaryiGQHMVulmDVXuQg
grimorio_bgTjTmQqAIK
grimorio_BsqXsGGLVZVEFYnCI
grimorio_DwIgcGqYWDHmmtCSZAHQhLkizAwf
grimorio_caFjJMeErFhst
grimorio_CfDLpWdDnqNpMrFHI
grimorio_cgFmEmNrkSB
grimorio_CoiTUDGnJfQotwZVsgN
grimorio_cuejJWfdyakNkhrNbbSuy
grimorio_CyEPFMBBFlaruUatsD
grimorio_daiDVdVkBx
grimorio_dJhVdVwUuAwVmlcwpTQoCTH
grimorio_ERQFPVhVhBfCSggtqwsE
grimorio_FlQqXSSSQ
grimorio_FddcEwKcF
grimorio_FzukkzbZlrTXAtwdr
grimorio_GbFwErgAGUGVlyIPiJiJUNDWkx
grimorio_gbHURvstFDQKsYcqsmaXh
grimorio_gqwLXUuivFZdJHzuuAmn
grimorio_GSAktwph
grimorio_hbPLGLdHKbMuJEOcKctLEwpaBpFK
grimorio_HladmuuMplsg
grimorio_igLaMqzhFFpJKXRJIMUJSZX
grimorio_ihvmmFuFqwyGOSZZJl
grimorio_IvXKJRURfNEfmaPeVST
grimorio_jrYUQHbzSAssJbfnq
grimorio_JsoutFgcPW
grimorio_jvAmSiAMQlgCKQOXdtW
grimorio_JzmMxjHpCkHLYRTwstsvHaU
grimorio_KDzBZHzCyIZbH
grimorio_KFMdOxYtGBHdcvHHTzXgxjRGQTSY
grimorio_KUAVfkvNAXASXpnszNKfQfhKzmBwk
grimorio_KwadfPriHGHRSyEdnJUVunEzwl
grimorio_kZrJLSSHrfseSztECuJjaolMm
grimorio_LbhtxWZGvgyeXkixwyuzMtp
grimorio_LmSZlqrDmkoDfMbLyNjWZuePuxGES
grimorio_LWRdHeHQQVGfN
grimorio_mgVJmDsr
grimorio_MHaujyWecCr
grimorio_mKbdDEUSyWHPSPecQLRRUqjzQ
grimorio_mLLOieSALQKsAm
grimorio_MlViabgVtBhODiSlw
grimorio_mRZazHRyOjC8juhXavaduH
grimorio_NAJzChunRJDikLicQylsd
grimorio_NcnxLSnGzrfcfjQtEuJ
grimorio_nRrtzgialiybokdHhdsYxr
grimorio_nTQXlYgGfGqKClGPMnciSlilWyh
grimorio_NvpuaulXMF
grimorio_OGBDjpZRIwgVppWTQwDhXio
grimorio_OzyPnuFRmgvmeMjpw
grimorio_OZyWpWjvt
grimorio_PffFUTDirJRBtRRQCLLmcfi
grimorio_PjQFRFwe
grimorio_pKVuWlJAlCtnYFwAHwPNAqQS
grimorio_pocGSZBwHGTZwGvckptaZoDZ
grimorio_PQVcuGbskbob
grimorio_pRULaiaXpWEPwhcYPTguus
grimorio_pUwoPxrnhKMDlhyPcifjofJo
grimorio_PZmTvmBYUboeGFG
grimorio_qiITPlhMnFNweLaxcveqCoosKaHc
grimorio_quPYAXGjQJdQFHSqwxX
grimorio_QrLzXNZNRcXjXaEdH
grimorio_RgvliKzeLflvDKcqqJQ
grimorio_rIHdGEilCzZiwUVPZBmcBLMPsGbIRJ
grimorio_RMGJLMuEcfNZANLWIRixP
grimorio_RmvFybJQawlgnlwMgTs
grimorio_RXxsCdPldsz
grimorio_sFgPhxDnECLegUQAqgBSf
grimorio_SlBTAKqJZeZHOPTfqmZuIACyRBLSPR
grimorio_sTwBcMXOIh
grimorio_SvnSobvWBovfLdxGyk
grimorio_TnfhJwFjYbxxxIewHkaJbgB
grimorio_tObvFvBaizdMwQOeyjXAMARtW0
grimorio_UCOLedDyLScndzpbIpMnbpAYNXLpLA
grimorio_uFNswgfmMVVOORNTYDZ
grimorio_VieeFieKzgvSWtUPiEPyFg
grimorio_VjUUPiZiVLl3YcACvA
grimorio_VkyLDfFhGVLsvltuoYKUDUPJcEreIk
grimorio_VNxWwbFgwK
grimorio_vQFLWzNyShbcFlgLqnKrJHvVohInx
grimorio_vSjTDJAmvaoYbDx
grimorio_VxlttXexwXPEiOorTBlMbcqZ
grimorio_WAYEjEjBxGdATuX
grimorio_xEdsNqpfEEzpTswGB0AAIjkyw
grimorio_xheNhuIuicSQQtThueSzDscrP
grimorio_xhzRRFUSowCKtJaxQOTForEafvYV
grimorio_xlCQNsVopaVeULMoQhZ
grimorio_xmIambBbK
grimorio_yFLxakMOLRNubkOaNBvnuBp
grimorio_yHfVpuemJESDsHcEmQpULFPbSnu
grimorio_yLoAoCRowKXRNkrxgcSgJJoqn
grimorio_yMQSwtAZiXMX
grimorio_YQAOdmFmgUoFavUetz
grimorio_ZhbYhMAfjbJbIXbJusNAFW
grimorio_zHUEgNlCnrgFoIFvMkqxInPswthvV
grimorio_ziUmhBANoAVO
grimorio_ZMogJAhy
grimorio_zoTdTNktGI
```

grep filtra le righe per pattern (-i, -v inverte, -n, -h). wc conta: -l righe, -w parole, -c caratteri.

grep -v "PAGATO" registro | wc -l

Processi e segnali: ps, pstree, kill

ps aux elenca i processi, pstree ad albero. kill invia segnali: SIGTERM(15), SIGKILL(9), SIGSTOP/

```
~/Castello/Cantina
[mission 33] $ cd

[mission 33] $ ls
Castello/ Foresta/ Giardino/ Montagna/ Stalla/

[mission 33] $ cd Castello

~/Castello
[mission 33] $ ls
Cantina/ Grande_sala/ Osservatorio/ Palazzo_principale/ Torre_principale/

~/Castello
[mission 33] $ cd Palazzo_principale/

~/Castello/Palazzo_principale
[mission 33] $ ls
Biblioteca/ Primo_piano/ Sala_del_trono/

~/Castello/Palazzo_principale
[mission 33] $ cd Biblioteca/

~/Castello/Palazzo_principale/Biblioteca
[mission 33] $ ls
Come_imbrogliare_negli_esami Greco_Latino_e_altre_lingue_moderne Matematica_101 Ufficio_di_Merlino/

~/Castello/Palazzo_principale/Biblioteca
[mission 33] $ cat .Come_imbrogliare_negli_esami
Il libro 'Matematica_101' contiene tutte le risposte.
Copia tutte le sue righe per avere il massimo punteggio.

~/Castello/Palazzo_principale/Biblioteca
[mission 33] $ cat Matematica_101
7224
34
3154
3432
3103
704
42
31512
4606
3250
972
9409
8352
```

SIGCONT.

ps aux | grep merlin ; kill -9 PID

Job control: &, Ctrl-Z, jobs, fg, bg

& avvia in background; Ctrl-Z sospende; jobs elenca; fg/bg riprendono.

sleep 100 & ; jobs ; fg %1

Ricerca: find

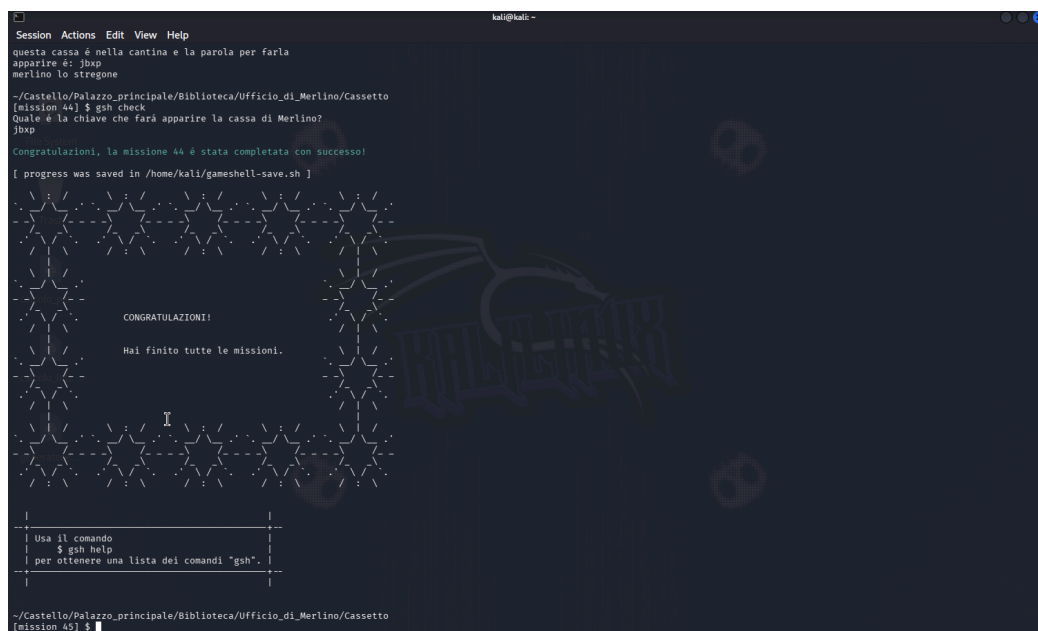
```
find . -name "*.log" -type f
```

tr (translate) sostituisce ogni carattere del primo insieme col corrispondente del secondo; utile per il cifrario di Cesare/ROT13. Con < riceve il testo da file.

Livelli preferiti

Missioni 37–39 — Quartiere dei Re (permessi). Sbloccare cartelle e file con `chmod`, capendo che su una cartella serve la `x` per entrarci, e scovare un biglietto nascosto con `ls -la`.

Missione 44 — Ufficio di Merlino (cifrario di Cesare). Decifrare con `tr`, scostamento 13 (ROT13):
`tr 'a-z' 'n-za-m' < messaggio_segreto`.



Parte 2 — Task 2: Attacco brute-force a SSH

Ambiente di laboratorio (Task 2)

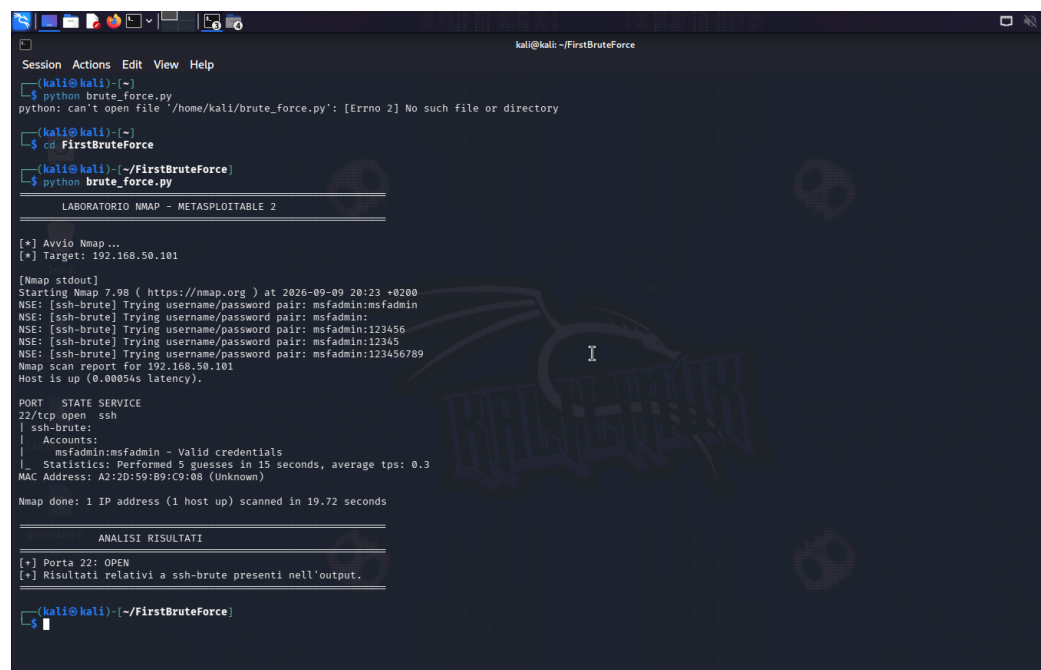
- Attaccante: Kali Linux (UTM), IP 192.168.50.100.
- Bersaglio: Metasploitable 2, IP 192.168.50.101, servizio SSH su porta 22.
- Rete: isolata (host-only) tra le due VM, senza alcuna esposizione a Internet.

Metodo

Il test è stato svolto interamente nel laboratorio isolato descritto sopra. È stato predisposto uno script in Python che invoca nmap con lo script NSE ssh-brute sulla porta 22 del bersaglio, ne raccoglie l'output e verifica l'esito. La prova è stata condotta in due fasi: una prima esecuzione con le liste di utenti e password predefinite di ssh-brute e una seconda in cui il dizionario utenti è stato ristretto, tramite un file userdb dedicato, al solo username corretto.

Risultati

Nella prima esecuzione, con le liste predefinite di ssh-brute, tutti i tentativi erano falliti (le relative tracce sono nei log della sezione Rilevamento). Restringendo il dizionario utenti al solo username corretto tramite il file userdb, la prova ha avuto esito positivo: lo script ha individuato la credenziale valida msfadmin:msfadmin. L'output di nmap riporta msfadmin:msfadmin - Valid credentials e le statistiche Performed 5 guesses in 15 seconds, con la scansione completata in circa 19,7 secondi. La porta 22 risulta open. La schermata seguente mostra l'esito:



```
kali@kali: ~/FirstBruteForce
Session Actions Edit View Help
kali@kali:~$ python brute_force.py
python: can't open file '/home/kali/brute_force.py': [Errno 2] No such file or directory

kali@kali:~$ cd FirstBruteForce
kali@kali:~/FirstBruteForce$ python brute_force.py

LABORATORIO NMAP - METASPLOITABLE 2

[*] Avvio Nmap...
[*] Target: 192.168.50.101

[Nmap stdout]
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-09 20:23 +0200
NSE: [ssh-brute] Trying username/password pair: msfadmin:msfadmin
NSE: [ssh-brute] Trying username/password pair: msfadmin:
NSE: [ssh-brute] Trying username/password pair: msfadmin:123456
NSE: [ssh-brute] Trying username/password pair: msfadmin:12345
NSE: [ssh-brute] Trying username/password pair: msfadmin:123456789
Nmap scan report for 192.168.50.101
Host is up (0.00054s latency).

PORT      STATE SERVICE
22/tcp    open  ssh
| ssh-brute:
| Accounts:
|   msfadmin:msfadmin - Valid credentials
| Statistics: Performed 5 guesses in 15 seconds, average tps: 0.3
MAC Address: A2:2D:59:B9:C9:08 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 19.72 seconds

ANALISI RISULTATI
[*] Porta 22: OPEN
[*] Risultati relativi a ssh-brute presenti nell'output.

kali@kali:~/FirstBruteForce$
```

L'efficacia di un brute-force dipende quasi interamente dalla robustezza della password. Con una credenziale debole o nota — come in questo ambiente didattico — la ricerca ha esito rapido (pochi secondi); con una password lunga e casuale, lo stesso approccio diventa impraticabile per tempo.

Rilevamento dell'attacco

Durante i tentativi, il bersaglio registra ogni accesso fallito in /var/log/auth.log, con righe del tipo Failed password for <utente> from <IP>. Gli indicatori tipici di un brute-force sono: molti tentativi falliti ravvicinati, provenienti dallo stesso IP, spesso su utenti comuni (root, admin). Sul bersaglio, per osservarli in tempo reale e quantificarli:

```
sudo tail -f /var/log/auth.log
```

Le schermate seguenti, catturate sul bersaglio durante la prima esecuzione (con le liste predefinite di ssh-brute), mostrano proprio queste tracce: Failed none for invalid user, Failed password for root e pam_unix(sshd:auth): authentication failure ... rhost=192.168.50.100, tutte ravvicinate e provenienti dall'IP dell'attaccante (Kali, 192.168.50.100).

```
Sep  9 14:07:01 metasploitable sshd[5317]: Failed none for invalid user  
m 192.168.50.100 port 60594 ssh2  
Sep  9 14:07:01 metasploitable sshd[5317]: pam_unix(sshd:auth): check pa  
unknown  
Sep  9 14:07:01 metasploitable sshd[5317]: pam_unix(sshd:auth): authenti  
ailure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.50.100  
Sep  9 14:07:01 metasploitable sshd[5316]: pam_unix(sshd:auth): authenti  
ailure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.50.100 user=  
Sep  9 14:07:01 metasploitable sshd[5320]: Invalid user admin from 192.1  
0  
Sep  9 14:07:01 metasploitable sshd[5320]: Failed none for invalid user  
om 192.168.50.100 port 60614 ssh2  
Sep  9 14:07:01 metasploitable sshd[5320]: pam_unix(sshd:auth): check pa  
unknown  
Sep  9 14:07:01 metasploitable sshd[5320]: pam_unix(sshd:auth): authenti  
ailure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.50.100  
Sep  9 14:07:02 metasploitable sshd[5317]: Failed password for invalid u  
from 192.168.50.100 port 60594 ssh2  
Sep  9 14:07:02 metasploitable sshd[5316]: Failed password for root from  
.50.100 port 60580 ssh2  
Sep  9 14:07:02 metasploitable sshd[5320]: Failed password for invalid u  
n from 192.168.50.100 port 60614 ssh2  
Sep  9 14:07:03 metasploitable sshd[5314]: Failed password for invalid u  
from 192.168.50.100 port 60578 ssh2
```

```
Sep  9 14:10:07 metasploitable sshd[5587]: Failed none for invalid user  
rator from 192.168.50.100 port 41678 ssh2  
Sep  9 14:10:07 metasploitable sshd[5588]: Failed none for invalid user  
from 192.168.50.100 port 41692 ssh2  
Sep  9 14:10:07 metasploitable sshd[5594]: Failed none for invalid user  
from 192.168.50.100 port 41734 ssh2  
Sep  9 14:10:07 metasploitable sshd[5588]: pam_unix(sshd:auth): check pa  
unknown  
Sep  9 14:10:07 metasploitable sshd[5588]: pam_unix(sshd:auth): authenti  
ailure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.50.100  
Sep  9 14:10:07 metasploitable sshd[5592]: pam_unix(sshd:auth): check pa  
unknown  
Sep  9 14:10:07 metasploitable sshd[5592]: pam_unix(sshd:auth): authenti  
ailure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.50.100  
Sep  9 14:10:07 metasploitable sshd[5594]: pam_unix(sshd:auth): check pa  
unknown  
Sep  9 14:10:07 metasploitable sshd[5594]: pam_unix(sshd:auth): authenti  
ailure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.50.100  
Sep  9 14:10:07 metasploitable sshd[5587]: pam_unix(sshd:auth): check pa  
unknown  
Sep  9 14:10:07 metasploitable sshd[5587]: pam_unix(sshd:auth): authenti  
ailure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.50.100  
Sep  9 14:10:08 metasploitable sshd[5585]: Failed password for root from  
.50.100 port 41668 ssh2
```

Questi tentativi appartengono alla prima esecuzione, che usava la lista utenti predefinita di ssh-brute, da qui il rumore e i numerosi fallimenti. Nella seconda esecuzione, con il userdb ristretto al solo username corretto, i tentativi si riducono a pochi e l'attacco va a segno.